Lab03. Using Code Commit to keep your code

Creation and Administration of Cloud-based Applications

Wojciech Thomas

Fall 2026

1 Learning objectives

1. Use CodeCommit service to keep a central Git repository

2 Exercises

2.1 Start

1. Start your AWS Learning Lab, and open AWS Console.
2. Start `devserver` environment created in the previous lab.

2.2 Working with CodeCommit

1. Open AWS Console and select CodeCommit service.
2. Click button “Create repository”.
3. Set the name of repository to `flask-app-1` and click Create button.
4. Click Clone URL list and select Clone HTTPS. Paste it in the Notepad.
5. Switch to the `devserver` window. In the Terminal (bottom), change folder to your folder including file `Dockerfile`: eg. `cd demo`
6. Create a Git repository in the folder:

```
git init
git config user.name "<YOUR NAME>"
git config user.email "<YOUR@EMAIL>"
```

7. Create file `.gitignore` to ignore virtual Python environment (if your virtual environment is in the other folder, use your folder name):

```
git status
echo ".venv/" >> .gitignore
git status
```

   Compare the first and the second status, and verify if the folder `.venv` is now ignored.

8. Create a new file `README.md`. Put your first and last name in the first line.
9. Commit all your files in the local repository:

```
git add .
git commit -m "Initial commit"
```

10. Configure remote repository:

    ```
    git remote add origin <CLONED HTTPS URL>
    git remote -v
    ```

    The command should look like:

    ```
    git remote add origin
     ↪  https://git-codecommit.us-east-1.amazonaws.com/v1/repos/flask-app-1
    ```

11. In AWS you will use a credential helper to retrieve credentials to access Git repository. Run the following commands to configure credential helper:

    ```
    git config --global credential.helper '!aws codecommit credential-helper
     ↪  $@'
    git config --global credential.UseHttpPath true
    ```

12. Send your code to the remote repository: `git push origin master`
13. Make some changes in your code. Commit each change separately.
14. Display repository log: `git log`, and observe where is HEAD, and where is `origin/master`.
15. Perform command `git commit`, and repeat previous task. Can you spot the difference?
16. Try to push your changes to CodeCommit: `git push` (compare this command to the previous `push`). This command fails, as Git does not know what to push.
17. Define persistently what should be pushed to CodeCommit:

    ```
    git push --set-upstream origin master
    ```

18. Commit some changes, and observe, that now it is enough to run `git push` to send current changes to CodeCommit.
19. Open CodeCommit and watch the state of the repository. Does it contain your code in the current version?
20. **Before the next step make sure that CodeCommit contains current version of your code!!!**
21. In the Terminal change directory to `projects` directory and delete your project:

    ```
    cd ..
    rm -rf <YOUR PROJECT FOLDER>
    ```

22. Clone your repository from CodeCommit:

    ```
    git clone <CLONED HTTPS URL>
    ```

23. Open cloned repository and verify if it contains your project. Pay attention that now project folder has the same name as remote repository. Review the configuration of the cloned repository.

    ```
    cd flask-app-1
    ls
    git config --list
    ```

    Check if the username and email are set. If not - configure them.

24. Make some changes to any file, but **do not** commit them in Bash.
25. Press Ctrl-Shift-G or select Source Control in left icon pane.
26. Look at Changes section. It should contain the list of changed files.
27. Click + (next to Changes) to Stage modified files (it executes command `git add`).

28. Enter commit comment in the Message field and press Ctrl-Enter to commit changes.

2.3 About commit messages

When you create a commit, you have to give it a name: `git commit -m "Another commit"`. Using meaningful name is crucial, when you care about your code. There are some rules that help you to create good commits' names:

1. The first line should be in present tense, e.g.: `create a file`
2. The first line should start with one of common prefixes, e.g.: `feat:` (for new features), `fix:` (for commits with fixes), `doc:` (if you add just documentation in the commit), `chore:` (if you change your configuration and not the code) etc.
3. The first line should be no longer than 50 characters.
4. If you need a longer comment, put it after the empty line. The comment should contain the intention or justification of the commit. E.g.:

```
git commit -m "fix: add URL verification

In some cases empty URL crashes application"
```